AlovLab

ALOVLAB ACADEMY · МЕТОДИЧКА К РИЛСУ «МОНТАЖ ТЕПЕРЬ ДЕЛАЕТ КЛОД»

Монтаж с AI-агентом

Claude Code и Codex: какой репозиторий, какие скиллы, какая роль и контекст у агента, чтобы он сам монтировал твои ролики.

Батч 1 · главы 1-4

Claude Code + Codex CLI

СОДЕРЖАНИЕ

Батч 1 из N

Это главы 1-4 из планируемых ~13: что создаём, что подготовить, репозиторий под монтаж, первый skill. Дальше — права доступа и hooks (реальная защита исходников), отдельный subagent, Codex CLI с нуля, сравнение инструментов, финальный проект, бонусы — следующими батчами. Оглавление кликабельно.

01 Что мы создаём 03
02 Два уровня 04
03 Что нужно подготовить 05
04 Структура папок 06
05 CLAUDE.md: что туда писать 07
06 CLAUDE.md: три практических момента 08
07 Skill «video-editor»: собираем 09
08 Skill «video-editor»: готовый пример 10
09 Skill «video-editor»: добавляем скрипты 11

ГЛАВА 1

Что мы создаём

К концу этого учебника у тебя будет репозиторий, в котором AI-агент сам режет клипы, синхронит голос, добавляет титры и собирает готовый файл по твоей команде.

Раньше монтаж ролика — это ты сидишь в программе-редакторе и вручную тянешь дорожки, режешь по кадрам, подставляешь титры. Здесь другой подход: ты даёшь задачу текстом («вырежи с 10 по 25 секунду, подставь голос, добавь титры снизу»), а всю черновую механику выполняет AI-агент через обычные консольные инструменты (в первую очередь ffmpeg). Ты проверяешь результат и правишь то, что реально не так.

твоя команда текстом → **агент (Claude Code / Codex)** → **ffmpeg режет/собирает** → **готовый файл**

Два инструмента в этом учебнике

CLAUDE CODE

Терминальный агент от Anthropic. Основной инструмент этого учебника — умеет читать репозиторий, запускать команды (в том числе ffmpeg), помнить правила проекта через файл CLAUDE.md, подключать Skills (готовые наборы инструкций под задачу).

CODEX CLI

Терминальный агент от OpenAI. Разбираем во второй половине учебника как альтернативу/сравнение — похожая идея (свой файл памяти, свои режимы доступа к системе), но другие названия и детали. Пригодится, если уже пользуешься им или хочешь сравнить.

Дальше — два уровня работы с этим учебником и то, чего автоматизация монтажа НЕ заменяет.

ГЛАВА 1

Два уровня в этом учебнике

УРОВЕНЬ 1 — НОВИЧОК

Просто общаешься с агентом в чате: «вырежи этот кусок», «добавь титр здесь». Агент сам решает, какие команды выполнить. Минимальная настройка репозитория.

УРОВЕНЬ 2 — ПРОДВИНУТЫЙ

Готовый Skill с набором проверенных операций, настроенные права доступа, чтобы агент не спрашивал разрешения на каждый чих, и выбор — подложить отдельного subagent под монтаж. Для тех, кто делает это каждую неделю.

ЧЕГО ЭТО НЕ ЗАМЕНЯЕТ

Это не нейросеть, которая «снимает» видео с нуля — это автоматизация механической работы над уже готовым материалом. Что вырезать и в каком порядке — решаешь ты, агент выполняет.

Агент не «знает» твой проект сам по себе — правила описываешь один раз в файле CLAUDE.md, и дальше он читает их в каждой сессии. Как именно — глава 3.

ГЛАВА 2

Что нужно подготовить

Прежде чем настраивать репозиторий, собери всё по списку ниже.

- [ ] Компьютер (Mac, Linux, или Windows через WSL2 — для Codex CLI это обязательное условие, Claude Code работает и без WSL).
- [ ] Установленный Claude Code (claude.ai/code) или Codex CLI (или оба — сравнение в главе 9).
- [ ] Установленный ffmpeg — бесплатный консольный инструмент для резки/сборки видео, без него ни один из агентов монтировать не сможет.
- [ ] Папка проекта с исходными материалами: отснятые клипы, голосовая дорожка, текст титров.
- [ ] git — не обязателен для самого монтажа, но полезен, чтобы не потерять настройку репозитория (см. главу 3).

ПРОВЕРЬ ВЕРСИИ ПЕРЕД ТЕМ КАК ДОВЕРЯТЬ ДЕТАЛЯМ ИЗ ЭТОЙ МЕТОДИЧКИ

Оба инструмента быстро меняются. Перед работой выполни `claude --version` или `codex --version` и сверь с официальной документацией — часть механизмов в этой методичке привязана к конкретным версиям, это явно отмечено в тексте.

Проверка ffmpeg

1. Открой терминал и выполни `ffmpeg -version`.
2. Если команда не найдена — установи через менеджер пакетов своей системы (на Mac через Homebrew, на Linux — через apt/dnf, на Windows — через WSL2 или сборку с официального сайта ffmpeg).
3. Проверь, что агент тоже видит ffmpeg: в чате Claude Code или Codex попроси «выполни ffmpeg -version» — если команда выполнится и покажет версию, всё готово к главе 3.

ГЛАВА 3.1

Структура папок

Прежде чем агент тронет хоть один файл, реши, где что лежит. Это не формальность — именно эта структура защищает исходники в главе 5.

```
reels-edit/                    ← корень репозитория
CLAUDE.md                      ← правила проекта (глава 3.2)
.gitignore                     ← исключает тяжёлые папки из git
.claude/
  settings.json                ← права доступа, общие для команды (глава 5)
  settings.local.json          ← личные права, не в git (глава 5)
  skills/
    video-editor/
      SKILL.md                 ← сам skill (глава 4)
      reference.md             ← шпаргалка по командам ffmpeg
      scripts/                 ← готовые обёртки над ffmpeg
footage/                       ← исходные клипы, НИКОГДА не правятся
voiceover/                     ← аудиодорожки озвучки
scenes/                        ← текстовые планы монтажа (что откуда куда)
work/                          ← промежуточные файлы, можно удалять
exports/                       ← готовые файлы на выдачу
```

FOOTAGE/

Сырые материалы. По правилу проекта — только читаются, никогда не перезаписываются. Это защищает тебя от ситуации «агент случайно затёр оригинал».

EXPORTS/

Только готовые файлы на выдачу, с понятными именами (дата + название ролика), не `final2.mp4`.

Эта структура — не единственно верная, но рабочая отправная точка. Дальше в этой главе — что писать в CLAUDE.md, чтобы агент эту структуру соблюдал.

ГЛАВА 3.2

CLAUDE.md: что туда писать

CLAUDE.md — файл в корне репозитория, который Claude Code читает автоматически в начале каждой сессии. Это и есть «контекст и роль» агента для этого проекта.

CLAUDE.MD (ПРИМЕР ДЛЯ ЭТОГО ПРОЕКТА)

```
# Монтаж Reels

Роль: ты монтажёр-ассистент. Выполняешь задачи по монтажу текущего ролика, не
придумываешь креатив сам — решения о порядке кадров и смысле принимает
человек.

Структура: footage/ — исходники, НИКОГДА не перезаписывай и не удаляй файлы в
этой
папке. voiceover/ — аудиодорожки. scenes/ — текстовые планы монтажа. work/ —
промежуточные файлы, можно перезаписывать свободно. exports/ — готовые файлы,
называй их <дата>_<название>.mp4.

Инструменты: используй ffmpeg через Bash для всех операций с видео. Перед
сборкой
финального файла прогоняй ffprobe и сообщай длительность и разрешение
результата.
```

ВАЖНО: CLAUDE.MD — ЭТО КОНТЕКСТ, НЕ ОХРАНА

По официальной документации Claude Code, CLAUDE.md передаётся агенту как обычное сообщение — агент старается следовать написанному, но это не жёсткое ограничение на уровне системы. Фраза «никогда не перезаписывай footage/» в CLAUDE.md — это сильная подсказка, а не гарантия. Настоящая защита — права доступа и hooks из главы 5. Не путай «попросил вежливо» с «запретил технически».

ГЛАВА 3.3

CLAUDE.md: три практических момента

Размер. Официальная рекомендация — держать файл компактным (ориентир до ~200 строк), с разделами по заголовкам. Длинный файл агент читает хуже, не лучше.

Личные заметки. Если хочешь что-то только для себя, а не для всей команды — используй отдельный `CLAUDE.local.md` и добавь его в `.gitignore` — общий CLAUDE.md в git видит вся команда.

Проверка. Команда `/context` внутри Claude Code показывает, какие файлы памяти реально загружены в этой сессии — полезно, если кажется, что агент «не слушается» написанного.

Импорты. Если хочешь вынести шпаргалку по ffmpeg в отдельный файл, подключи её строкой `@scenes/ffmpeg-cheatsheet.md` прямо в CLAUDE.md — Claude Code подхватит её при чтении.

Дальше — сам skill «video-editor»: готовая инструкция, которую Claude Code подключает сам, когда видит подходящий запрос.

ГЛАВА 4.1

Skill «video-editor»: собираем

Skill — это папка с готовой инструкцией под конкретную задачу, которую Claude Code подключает сам, когда видит подходящий запрос. Разбираем по схеме: где находится → что срабатывает → что внутри → что вводим.

ГДЕ НАХОДИТСЯ. Файл `.claude/skills/video-editor/SKILL.md` в корне репозитория (личный skill, доступный только тебе на этой машине, живёт в `~/.claude/skills/` — но для проекта с командой правильнее держать его в репозитории).

ЧТО СРАБАТЫВАЕТ. Поле `description` в самом верху SKILL.md. Claude Code сравнивает твой запрос с этим описанием и сам решает, подключать ли skill — или ты вызываешь его вручную командой `/video-editor`.

ЧТО ВНУТРИ. YAML-заголовок (между строк `---`) с полями name/description/allowed-tools, а дальше обычный текст-инструкция для агента. Рядом может лежать папка `scripts/` с готовыми обёртками команд.

ЧТО ВВОДИМ. Описание задачи простыми словами + список разрешённых команд (allowed-tools) + текст-инструкция, что куда писать и чего не трогать. Пример — в следующей главе.

Дальше — полный пример такого SKILL.md целиком, готовый к копированию.

ГЛАВА 4.2

Skill «video-editor»: готовый пример

SKILL.MD (.CLAUDE/SKILLS/VIDEO-EDITOR/SKILL.MD)

```
---
name: video-editor
description: Монтирует вертикальное короткое видео через ffmpeg. Используй,
когда
  просят вырезать клип по таймкодам, склеить несколько клипов, добавить
  титры/подпись, подставить дорожку озвучки или собрать финальный файл 9:16.
allowed-tools:
  - Bash(ffmpeg *)
  - Bash(ffprobe *)
---

Исходники в footage/ — никогда не изменяй и не перезаписывай.
Промежуточные файлы пиши в work/, финальные — в exports/.
Перед сборкой финального файла прогони ffprobe и сообщи длительность и
разрешение.
```

allowed-tools здесь — это подсказка самому Claude Code, какими командами пользоваться в рамках skill; жёсткие правила доступа (чтобы не спрашивать разрешения на каждый запуск ffmpeg) настраиваются отдельно в settings.json — это глава 5.

ГЛАВА 4.3

Skill «video-editor»: добавляем скрипты

Вместо того чтобы агент каждый раз сам придумывал ffmpeg-команду с нуля, можно положить рядом готовые проверенные скрипты — это снижает риск ошибки в сложном синтаксисе фильтров.

TRIM.SH
Вырезает кусок клипа по таймкодам начала и конца — одна из самых частых операций.

CONCAT.SH
Склеивает несколько клипов подряд в один файл.

OVERLAY_TEXT.SH
Накладывает текстовую подпись/титр поверх видео в заданной позиции и с заданным временем появления.

MUX_VOICE.SH
Подставляет отдельную аудиодорожку озвучки поверх видео, синхронизируя по длительности.

Как агент их находит. В тексте SKILL.md скрипты называются через переменную `${CLAUDE_SKILL_DIR}` — она разворачивается в путь до папки самого skill, так что ссылка остаётся верной независимо от того, где именно лежит репозиторий на твоём компьютере.

SKILL.MD — ССЫЛКА НА СКРИПТ

```
Для вырезания фрагмента запускай:
${CLAUDE_SKILL_DIR}/scripts/trim.sh <входной_файл> <начало> <конец>
<выходной_файл>

Пример: ${CLAUDE_SKILL_DIR}/scripts/trim.sh footage/clip-01.mp4 00:00:10
00:00:25 work/clip-01-trim.mp4
```

ЧЕСТНО ПРО ЭТУ МЕТОДИЧКУ

Сами файлы `trim.sh` / `concat.sh` и остальные — не вложены как готовые бинарные файлы в этот PDF. Содержимое каждого — это одна строка вызова ffmpeg с понятными параметрами (обрезка по времени, склейка через список файлов, наложение текста фильтром drawtext, замена звуковой дорожки). Собрать их самостоятельно по документации ffmpeg — следующий бонусный блок методички (глава 13, бонусы), либо попроси агента написать их за тебя по этому же описанию задачи.